

INSTITUT TEKNOLOGI DEL

EXCEPTION HANDLING

Fakultas Vokasi | D4TRPL | Institut Teknologi Del

Week 7 Sesi 1 - CDN - Tahun Ajaran 2026/2027

Konten

- Exception Concepts
- Exception Hierarchy
- Try, catch, and finally
- Checked and Unchecked Exception
- Throw and Throws
- Creating User-defined Exception

Filosofi Exception Handling

Exception Handling pada Java dibangun di atas filosofi **“controlled failure”,** yang memberlakukan pendekatan terstruktur untuk peristiwa tak terduga daripada mengizinkan *error codes* atau *silent failures*. Hal ini didasarkan pada dua prinsip utama:

- **Fail-fast**: Masalah diidentifikasi dan ditangani sedini mungkin untuk mencegahnya masalah meningkat menjadi masalah yang lebih besar.
- **Separation of Concerns**: Logika bisnis sebuah program tetap berbeda dari pemulihan kesalahan, sehingga membuat kode lebih bersih dan lebih mudah dikelola.

Prinsip-prinsip ini memastikan bahwa exception tidak hanya ditangani, tetapi secara aktif dikelola untuk meningkatkan keandalan sistem.

Konsep Dasar Exception

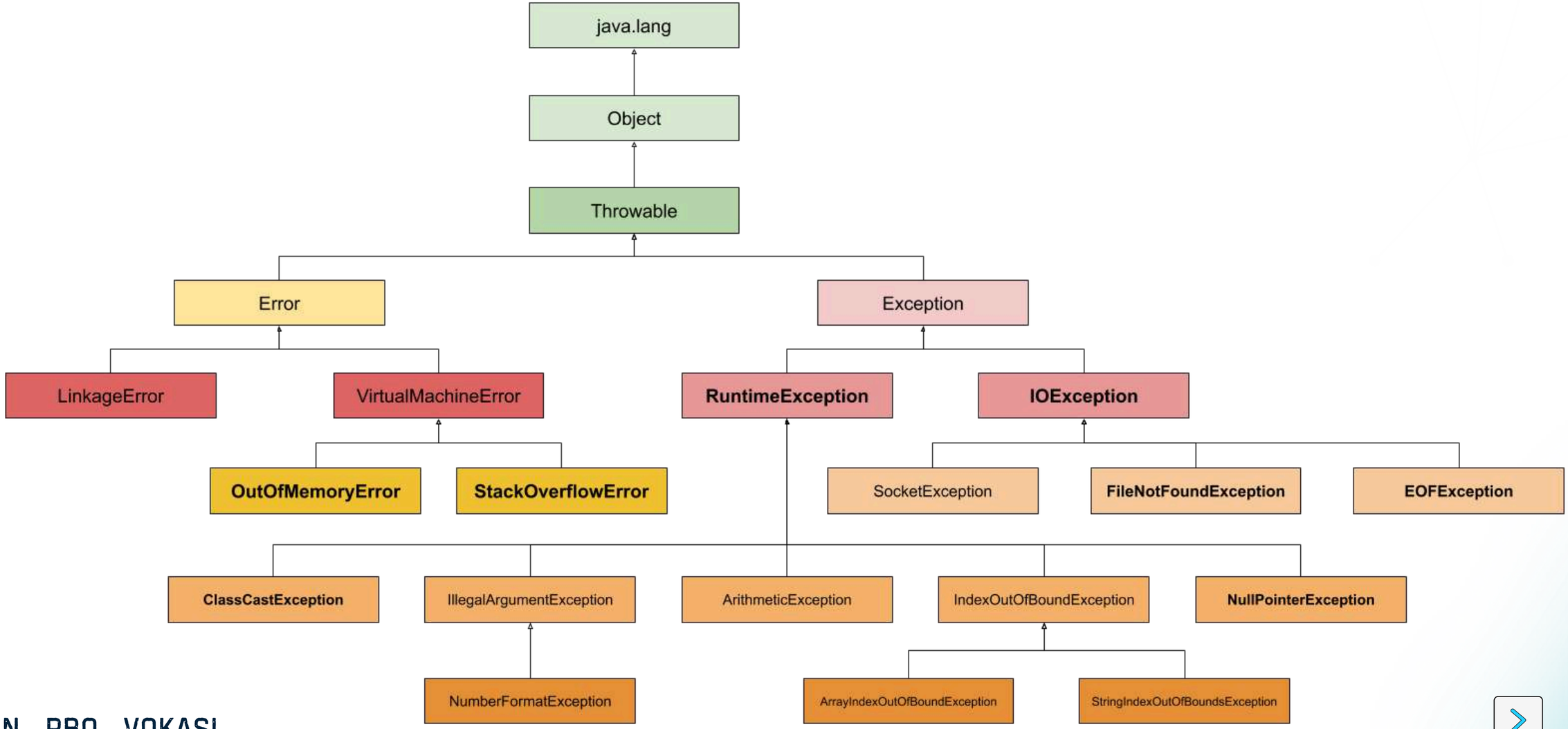
Exception handling *is a cornerstone of robust software development in Java, serving as the bridge between theoretical correctness and practical resilience.*

- Exception merupakan **kejadian tidak normal** atau **salah** saat program berjalan.
- Tanpa exception handling → program berhenti ketika error.
- Dengan exception handling → program bisa menangani error dan melanjutkan proses berjalan-nya program.

Prinsip utama-nya adalah:

Kode yang salah tidak boleh dijalankan, tapi harus ditangani dengan aman.

Exception Hierarchy



Exception Hierarchy

Hierarki exception pada Java berakar pada kelas **Throwable**, dengan tiga kategori utama:

- ***Errors*** (*Error subclasses*):
  - Severe, non-recoverable issues (e.g., OutOfMemoryError, StackOverflowError).
  - Typically arise from JVM or system-level failures.
- ***Checked Exceptions*** (*Exception subclasses*):
  - Enforced by the compiler (e.g., IOException, SQLException).
  - Represent recoverable errors (e.g., file not found, network issues).
  - Require explicit handling via try-catch or propagation using throws.
- ***Unchecked Exceptions*** (*RuntimeException subclasses*):
  - Not enforced by the compiler (e.g., NullPointerException, IllegalArgumentException).
  - Often indicate programming errors (e.g., invalid arguments, logic flaws).

Handling Exceptions

Untuk menangani exceptions, Java menyediakan 2 pendekatan yang memungkinkan yaitu:

1. JVM Default Handling

Jika pengecualian tidak ditangani secara eksplisit, maka umumnya JVM bertanggung jawab untuk menangani pengecualian tersebut.

```
import java.util.*;
class Main {
   public static void main (String[] args) {
     System.out.println(5/0);
     System.out.println("End of program!");
   }
}
```

```
Runtime Error:
Exception in thread "main"
java.lang.ArithmeticException: / by zero
at Main.main(File.java:5)
```

Handling Exceptions

Untuk menangani exceptions, Java menyediakan 2 pendekatan yang memungkinkan yaitu:

2. Developer Handling

Pengembang dapat secara eksplisit menulis implementasi untuk menangani pengecualian. Setelah pengecualian ditangani, eksekusi kode normal akan dilanjutkan.

Preferable: tangani exceptions untuk memastikan kode dapat dieksekusi secara normal.

```
try {
   // code that may throw an exception
} catch (ExceptionType e) {
   // code to handle the exception
} finally {
   // code that will always execute
}
```

```
Compile-time error:
prog.java:5: error: 'try' without 'catch',
'finally' or resource declarations
    try{
    ^
1 error
```

Handling Exceptions

- Penggunaan *exception handling* pada Java
  - Memproses eksepsi dari komponen program
  - Menangani eksepsi secara seragam dalam proyek besar
  - Menghapus kode penanganan kesalahan dari "jalur utama" eksekusi
  - Mencegah penghentian program yang tidak normal.
- Dikelola melalui lima kata kunci: `try`, `catch`, `throw`, `throws`, dan `finally`.
- Kode yang dapat menghasilkan kesalahan ditempatkan di blok **try**
  - Kode untuk penanganan kesalahan ditutup dalam klausa **catch**
  - Klausa **finally** selalu dieksekusi.
- Model terminasi exception handling
  - Blok di mana eksepsi terjadi berakhir.
- Klausa **throws** menentukan eksepsi yang dilempar metode.

Simple Exception

// Ini akan dikompilasi tetapi tidak dijalankan

```
public class Test{
    public static void main(String[] args)
    {
        System.out.println(3/0); // Akan menyebabkan ArithmeticException
    }
}
```

```
ERROR!
/tmp/SZ3wQwXWrM/Main.java:10: error: duplicate class: Test
public class Test{
       ^
1 error
ERROR!
error: compilation failed
```

```
//Solusi
public class Test{
    public static void main(String[] args) {
        try{
            System.out.println(3/0);
        }
        catch(Exception ex){
            System.out.println("Error: " + ex.getMessage());
        }
    }
}
```

An exception can occur for many reasons. Some of them are:

- Invalid user input
- Device failure
- Loss of network connection
- Physical limitations (out of disk memory)
- Code errors
- Opening an unavailable file

Try, Catch, dan Finally

Cara paling sederhana dan mendasar untuk menangani pengecualian adalah dengan menggunakan blok try – catch.

Struktur Try-Catch-Finally:

```
try {
  // kode yang potensi error
} catch (ExceptionType e) {
  // tangani error
} finally {
  // selalu dijalankan
}
```

- Jalankan blok pernyataan **try** sampai either sebuah eksepsi dilempar atau blok selesai dengan sukses.
- Jika eksepsi dilempar, klausa **catch** diperiksa secara berurutan, untuk menemukan satu untuk eksepsi dari kelas yang dilempar atau salah satu kelas induk dari kelas yang dilempar.

Bagaimana jika urutan dibalik?

```
try {
    int[] arr = new int[3];
    System.out.println(arr[5]);
} catch (Exception e) {
    System.out.println("Kesalahan umum.");
} catch (ArrayIndexOutOfBoundsException e) {
    System.out.println("Index error!");
}
```

- Blok **finally** selalu dieksekusi, bahkan jika tidak ada eksepsi yang dilempar. (blok finally dijalankan terlepas apakah try melemparkan eksepsi atau tidak).

Kode di atas akan menyebabkan error kompilasi karena , karena apa?

Try, Catch, dan Finally

```
public class Test{
    public static void main(String[] args){
        int i = 1;
        int j = 0;
        try{
            System.out.println("Try Block entered");
            System.out.println(i / j);
            // Akan melempar ArithmeticException
            System.out.println("Ending Try Block");
            // Baris di atas TIDAK akan dijalankan
        }
        ...
        ...
        catch(ArithmeticException ex) {
            System.out.println("Exception Caught: " +
            ex.getMessage());
        }
        finally {
            System.out.println("Finally block executed");
        }
        System.out.println("After try-catch-finally");
        // Baris di atas akan dijalankan
    }
}
```

Checked Exception

- Harus ditangani jika mereka pernah dilempar
  - Gunakan blok try-catch/throws
- Berurusan dengan masalah yang terjadi di tempat tertentu (external factors)
  - Saat metode tertentu dipanggil
- Contoh: IOException

Checked Exceptions:

Coba perhatikan dan identifikasi! kode di samping ini termasuk dalam **checked exception** karena apa?

```java
import java.io.*;

public class MultiCatchDemo {
  public static void main(String[] args) {
    try {
        BufferedReader br = new BufferedReader(new FileReader("data.txt"));
        String text = br.readLine();
        int number = Integer.parseInt(text);
        System.out.println("Angka: " + number);
        br.close();
    } catch (FileNotFoundException e) {
        System.out.println("File tidak ditemukan!");
    } catch (NumberFormatException e) {
        System.out.println("Format angka tidak valid!");
    } catch (IOException e) {
        System.out.println("Terjadi kesalahan I/O.");
    } finally {
        System.out.println("Eksekusi selesai.");
    }
  }
}
```

Unchecked Exception

Kompiler pada Java tidak mengharuskan Anda untuk menanganinya jika eksepsi dilempar.

- Dapat terjadi kapan saja dalam program (tidak hanya untuk metode tertentu).
- Biasanya terjadi karena **kesalahan runtime fatal** yang berada di luar kendali.
- Gunakan pernyataan kondisional daripada model penanganan eksepsi (untuk mencegahnya terjadi).
- Contoh: NullPointerException, IndexOutOfBoundsException, ArithmeticException, dll.

Unchecked Exceptions:

Coba perhatikan dan identifikasi! kode di samping ini termasuk dalam **unchecked exception** karena apa?

```
public class NullExample {
  public static void main(String[] args) {
    try {
      String s = null;
      System.out.println(s.length());
    } catch (NullPointerException e) {
      System.out.println("Variabel belum diinisialisasi!");
    }
  }
}
```

Throw and Throws

Untuk sebuah program yang secara eksplisit (manual) melemparkan sebuah eksepsi, maka:

- **throw** adalah kata kunci untuk menunjukkan bahwa suatu eksepsi telah terjadi.
- **throw** biasanya menentukan satu operand.
- **throw** ThrowableInstance
  - Operand yang dilempar adalah sebuah objek dari kelas Exception atau subclass-nya.
- Ketika sebuah eksepsi dilempar, kendali keluar dari blok try dan melanjutkan ke handler catch yang sesuai setelah blok try.

```
throw new ExceptionType("Pesan Error");
```

Kapan sebaiknya kita melempar exception secara manual daripada menunggu error terjadi? Perhatikan contoh kode program di samping!

Throw and Throws

Contoh: throw

```
// Demonstrasi throw.
class ThrowDemo {
    static void demoproc() {
        try {
            throw new NullPointerException("demo");
            // harus menggunakan new keyword untuk membuat instance baru dari exception
        } catch(NullPointerException e) {
            System.out.println("Caught inside demoproc.");
            throw e; // melemparkan ulang (re-throw) exception
        }
    }

    public static void main(String args[]) {
        try {
            demoproc();
        } catch(NullPointerException e) {
            System.out.println("Recaught: " + e);
        }
    }
}
```

Throw and Throws

Jika suatu metode mampu melemparkan eksepsi yang tidak ditangani sendiri, maka ia harus menentukan perilaku ini sehingga pemanggil metode dapat melindungi diri mereka dari eksepsi tersebut.

- Dilakukan dengan menggunakan klausa **throws** dalam deklarasi metode.
  - *exception-list terdapat* dalam Kelas `ReflectiveOperationException: ClassNotFoundException, InstantiationException, IllegalAccessException, dan InvocationTargetException atau checked exceptions lainnya.`
- Klausa **throws** mendaftarkan semua eksepsi yang dapat dilempar metode (kecuali Error/RuntimeException yang unchecked).
  - `returnType method-name(parameter-list) throws exception-list {.....}`

throws tidak menangani error — hanya memberi tahu bahwa error bisa terjadi. Perhatikan contoh kode program di samping!

Throw and Throws

Contoh: throws

```
class ThrowsDemo {
    static void throwOne() throws IllegalAccessException {
        System.out.println("Inside throwOne.");
        throw new IllegalAccessException("demo");
    }

    public static void main(String args[]) {
        try {
            throwOne();
        } catch (IllegalAccessException e) {
            System.out.println("Caught " + e);
        }
    }
}
```

User-defined Exception

Seorang pengembang tidak dibatasi pada eksepsi yang ditentukan di Java. **Kita dapat membuat eksepsi kita sendiri (user-defined Exception).**

- Kelas eksepsi dapat menyediakan fungsionalitas apa pun yang ditentukan pengembang diperlukan. Itu bisa jadi cukup kompleks.
- Perhatikan bahwa semua eksepsi mewarisi dari kelas Java Exception. Ini ditunjukkan dengan menggunakan kata kunci extends.

```
class InvalidAgeException extends Exception {
    public InvalidAgeException(String message) {
      super(message);
    }
}
```

User-defined Exception

Berikut adalah contoh generik sederhana dari eksepsi buatan pengguna:

```
// 1. Mendefinisikan Eksepsi Baru
public class MyException extends Exception {
    // Konstruktor
    MyException(String msg) {
        super(msg); // Meneruskan pesan ke konstruktor induk (Exception)
    }
}
```

```
// 2. Menggunakan Eksepsi Buatan Sendiri
public class MyExceptionDemo {
    public static void foo() throws MyException {
        System.out.println("Throwing MyException from foo()");
        throw new MyException("Ini adalah eksepsi kustom dari foo()");
    }
    public static void main(String[] args) {
        try {
            foo();
        } catch (MyException e) {
            System.out.println("Caught: " + e.getMessage());
            e.printStackTrace(); // Untuk menampilkan stack trace
        }
    }
}
```

Studi Kasus Kompleks

```
class BalanceException extends Exception {
   public BalanceException(String message) { super(message); }
}

class Account {
   private double balance;
   public Account(double balance) { this.balance = balance; }

   public void withdraw(double amount) throws BalanceException {
     if (amount > balance) {
        throw new BalanceException("Saldo tidak mencukupi untuk penarikan " + amount);
     } else {
        balance -= amount;
        System.out.println("Penarikan berhasil. Sisa saldo: " + balance);
     }
   }
}
```

Coba!! Hubungkan dengan konsep class dan encapsulation. Tunjukkan bagaimana exception menjaga keamanan data dalam sistem.

Studi Kasus Kompleks

```
//lanjutan..
public class BankApp {
    public static void main(String[] args) {
        Account acc = new Account(50000);
        try {
            acc.withdraw(70000);
        } catch (BalanceException e) {
            System.out.println("Kesalahan: " + e.getMessage());
        } finally {
            System.out.println("Transaksi selesai diproses.");
        }
    }
}
```

Coba!! Hubungkan dengan konsep class dan encapsulation. Tunjukkan bagaimana exception menjaga keamanan data dalam sistem.

Kesimpulan

Dalam konsep Object-Oriented Programming, **exception handling penting** karena:

- Menjaga encapsulation: error ditangani dalam kelas yang relevan.
- Meningkatkan modularity: setiap class bertanggung jawab atas error-nya sendiri.
- Meningkatkan reusability: kode yang aman dari error bisa digunakan ulang.
- Meningkatkan reliability sistem.

Java mengatur exception dengan lima kata kunci :

- `try, catch, throw, throws, finally.`

Hierarki Exception di Java:

- *Throwable* adalah induk semua error dan exception.
- **Error**:
  - Terjadi jarang dan fatal (misal: kehabisan memori).
  - Biasanya tidak ditangani oleh program.
- **Exception**:
  - Disebabkan bug atau input tidak valid, dan harus ditangani.
  - Bisa checked atau unchecked.
    - Checked → harus ditangani dengan try-catch atau throws
    - Unchecked → boleh diabaikan

Penutup

Any Questions?

TERIMAKASIH

Stay Happy & Healthy